

PSF used to create the blur and the noise power (NP):

```
>> NP=V*prod(size(I));
>> [reg1 LAGRA] = deconvreg(Blurred_
    Noise,PSF,NP);
>>figure, imshow(reg1)
```

Fig. 8 shows the restored image.

Hough transform

The Hough transform is designed to identify lines and curves within an image. Using the Hough transform, you can find line segments and end-points, measure angles, find circles based on size, and detect and measure circular objects in an image.

In the following example we take an image, automatically detect circles or circular objects in it, and visualise the detected circles.

1. Read the image and display it. This example uses an image of circles of various colours:

```
>> I=imread('Image_3.png');
>>figure, imshow(I)
```

Fig. 9 shows the image.

2. Determine radius range for searching circles. A quick way to find the appropriate radius range is to use the interactive tool `imdistline`, which gives an estimate of the radii of various objects.

`h = imdistline` creates a Distance tool on the current axis. The function returns `h`—handle to an `imdistline` object.

Distance tool is a draggable, resizable line, superimposed on an axis, which measures the distance between two endpoints of the line. It displays the distance in a text label superimposed over the line. The tool specifies the distance in data units determined by `XData` and `YData` properties, which is pixels by default.

If you wrote the following command in MATLAB, Fig. 9 would change to that shown in Fig. 10:

```
>>d = imdistline
```

The line can be dragged to get the size of different circles.

In a nutshell, `imdistline` command creates a draggable tool that can be moved to fit across a circle and the numbers can be read to get an approximate estimate of its radius.

To remove the `imdistline` tool, use the function:

```
>>delete(d);
```

3. Consider the image shown in Fig. 11. The aim is to find the number of circles in the figure.

Quite evidently, there are many



Fig. 8: Restored image



Fig. 9: Image of circles of various colours read and displayed



Fig. 10: Image shown when interactive tool `imdistline` is written in MATLAB



Fig. 11: Image with a number of circles

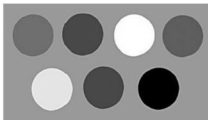


Fig. 12: RGB image in Fig. 11 converted into grayscale image

circles having different colours with different contrasts with respect to the background. Blue and red circles have strong contrast with respect to the background, whereas white and yellow circles have the poorest contrast with respect to the background.

4. We calculate the objects that are brighter and those that are darker than the background. For this, we need to convert this RGB image into grayscale version:

```
>> RGB=imread('Image_4.png');
>>gray_image=rgb2gray(RGB);
>>figure, imshow(gray_image)
```

Fig. 12 shows the grayscale image.

5. Function `centers = imfindcircles(A,radius)` finds circles in image `A` whose radii are approximately equal to `radius`. The output, `centers`, is a two-column matrix containing (x, y) coordinates of centers of the circles in the image.

`[centers,radii] = imfindcircles(A,radiusRange)` finds circles with radii in the range specified by `radiusRange`. The additional output argument, `radii`, contains the estimated radii corresponding to the center of each circle in `centers`.

By default, `imfindcircles` finds circular objects that are brighter than the background. In our case, let us set the parameter 'ObjectPolarity' to 'dark' in `imfindcircles` to search for dark circles:

```
>> [centers,radii]=imfindcircles(RGB,
    [40 60],'ObjectPolarity','Dark')
```

The result of the above command is:

```
centers = 384.2814 204.1827
radii = 50.1646
```

As you can see, only one circle is detected. This happens because `imfindcircles` is a circle detector, and similar to most detectors, `imfindcircles` has an internal detection threshold that determines its sensitivity. `imfindcircles` has a parameter 'sensitivity' that can be used to control this internal threshold, and consequently, the sensitivity of the algorithm. A higher sensitivity value sets a lower detection threshold and leads to detection of more circles. This is similar to the sensitivity control on motion detectors used in home security systems.

6. By default, sensitivity, which is a number between 0 and 1, is set to 0.85. Increase sensitivity to 0.9: